

Exploratory Data Analysis Bootcamp

Instructor Teaching Guide

Python Edition | For Facilitator Use | 21–29 August 2026

Day 1 — Foundations + Pandas

Date: 21 August 2026
(Fri)

Facilitator: Abdou
Salam Sisawo

Time: 5:30 PM

Venue: SCI Lab

Data Science & Python Basics

- **What Data Science actually is** — the blend of stats, programming, and domain knowledge; how it differs from “data analysis” or “software engineering”
- **Career roadmap / roles in the field**, with the tools each typically uses (brief, no deep dive):
 - Data Analyst — Excel, SQL, Power BI/Tableau
 - Data Scientist — Python (Pandas, Scikit-learn), R, SQL
 - Data/ML Engineer — Python, SQL, Spark, cloud tools (AWS, Airflow)
 - BI Analyst — Power BI, Tableau, SQL
- **Rough skill progression:** Excel/spreadsheets → SQL → Python (Pandas/NumPy/Matplotlib) → statistics → ML, tying today's bootcamp to the earlier Business Analytics Programme
- **Where EDA fits in the data science workflow** (collect → clean → explore → analyse → visualise → communicate)
- **Why EDA matters before any modelling**
- **Tools overview:** Python, Pandas, NumPy, Matplotlib
- Google Colab setup, running a first notebook
- **First taste of code:** `.head()`, `.shape()`, simple print statements, one basic stat

Pandas

- **Loading data:** `read_csv()` (and a note on `read_excel()`)
- **First-look commands:** `.head()`, `.tail()`, `.shape`, `.columns`, `.dtypes`, `.info()`
- **Selecting & indexing:** column selection, `.loc[]`, `.iloc[]`, boolean/conditional filtering
- **Sorting:** `.sort_values()`, `.sort_index()`
- **Descriptive stats & data quality:** `.describe()`, `.isnull().sum()`, `.value_counts()`, `.duplicated()` / `.drop_duplicates()`
- **Handling missing data:** `.fillna()`, `.dropna()`
- **Data type conversion:** `.astype()`
- **Creating & modifying** columns, renaming columns (`.rename()`)

- **Applying functions:** `.apply()` with lambda functions
- **String operations:** `.str` accessor (e.g. `.str.lower()`, `.str.contains()`)
- **Basic datetime handling:** `pd.to_datetime()`, extracting year/month/day
- **Grouping & aggregation:** `.groupby()`, `.mean()` / `.sum()` / `.count()`, multi-level grouping, sorting grouped results
- **Combining data:** `.merge()` / `.concat()` (**brief**)
- **Exporting:** `.to_csv()`
- **Guided practice:** business-style questions on a sustainability-themed dataset

Day 2 — NumPy + Matplotlib

Date: 22 August 2026
(Sat)

Facilitator: Fathima
Wazna

Time: 5:30 PM

Venue: SCI Lab

Recap of Day 1

- Quick recap of Data Science basics
- Resolve any outstanding setup/environment issues

NumPy

- What NumPy is and why it's faster than plain Python lists (vectorisation, no explicit loops)
- **Creating arrays:** `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`
- **Array indexing & slicing (1D and 2D)**
- **Reshaping arrays:** `.reshape()`
- **Vectorised math: element-wise +, -, *, / across arrays**
- **Boolean/fancy indexing:** `arr[condition]`, `arr[idx_array]`, `np.where()`
- **Sorting arrays:** `np.sort()`
- **Statistical functions:** `np.mean()`, `np.median()`, `np.std()`, `np.var()`
- **Percentiles/quartiles:** `np.percentile()`
- **Outlier detection:** Z-score method, IQR method
- **Correlation:** `np.corrcoef()`, reading a correlation matrix
- **Correlation vs. causation** (concept, not code)

Matplotlib

- **Why visualise:** turning numbers from the NumPy section into something people can actually read
- **Core chart types:** line chart, bar chart, histogram, scatter plot, boxplot, pie chart — when to use each
- **Customisation essentials:** titles, axis labels, legends, colours, markers, grid etc

- **Subplots:** showing multiple charts side by side (`plt.subplots()`)
- **Guided practice:** turning 2–3 of today's NumPy findings into labelled charts

Day 3 — Applied Practice & Capstone Kickoff

Date: 23 August 2026 **Facilitator:** Chit Ko Ko **Time:** 5:30 PM **Venue:** SCI Lab (Sun)

Dataset for this session: a business dataset (not sustainability-themed) — continues naturally from the earlier Business Analytics Programme and fits a business recommendation format.

Suggested 2-Hour Breakdown

- Recap (15 min) — walk through what's been covered: Day 1 (Data Science basics, Pandas) and Day 2 (NumPy, Matplotlib), framing it as “here's the full toolkit you now have”
- Guided Walkthrough (45 min) — live demonstration combining Pandas + NumPy + Matplotlib into one workflow on a sample business dataset (load → clean → explore → analyse → visualise), so students see the full process, not just individual tools
- Capstone Brief (10 min) — reveal the business dataset(s), explain the task and expected deliverable (a problem/pattern + a recommendation); brief note on team assignments (already handled separately)
- Independent Practice (40 min) — teams work alone on their dataset applying what they just saw; facilitator circulates to guide one-on-one, not lead
- Wrap-up (10 min) — quick check-in on progress, preview of Day 4 (presentations)

Day 4 — Capstone Presentations

Date: 29 August 2026 **Facilitator:** Chit Ko Ko **Time:** 5:30 PM **Venue:** SCI Lab (Sat)

- Final independent work time
- Team presentations (3–5 min each): findings + recommendation
- Peer input / informal judging
- Certificate distribution & group photo
- Closing remark

Instructor Notes

Please read before your session. These notes apply across all four days.

Pacing

- Watch the room, not the clock. If most students look lost, slow down and re-explain before moving on.
- Equally, don't over-linger on parts the class has clearly understood — time is tight, so keep momentum on the easy wins.
- It's fine to trim depth on a topic if you're running behind, but don't skip the guided practice — that's the part students learn the most from.

Context & Connection

- Always explain the “why” before the “how.” Before introducing a tool or function, briefly explain what problem it solves and where it fits in the overall EDA workflow.
- Make sure students understand the flow connecting the days — Day 1's Pandas work feeds into Day 2's NumPy/Matplotlib work, which feeds into Day 3's applied practice. Remind them of this link at the start of each session.
- If possible, sit in briefly on the other facilitators' sessions (even just 10–15 minutes) so you know what was actually taught and how, and can build on it consistently. The goal is for the bootcamp to feel like one connected programme, not four separate, disconnected sessions.

Practice

- Practice time is the most important part of each session — protect it, don't let it get cut short by running over on lecture content.
- During practice, guide students one by one rather than only presenting at the front. Walk around, check in with individuals or small groups.
- When helping a student, don't just fix the code for them — explain what the code does, why it's structured that way, and what each line/command is doing.

General Tips

- Live-code rather than only showing slides — watching code being written and explained in real time helps students follow the reasoning, not just the result.
- Check for understanding periodically (quick show of hands, or ask a student to explain a step back to you) rather than assuming silence means everyone's following.
- Use relatable, simple examples when introducing a new concept before moving to the session's main dataset.
- Encourage questions throughout, not just at the end — questions early often prevent confusion from compounding later in the session.